Analiza projekta Guess My Drawing

Jelena Ivanović, 1133/25
Verifikacija softvera
Matematički fakultet, Univerzitet u Beogradu

Februar 2026

Sadržaj

1 Uvod **3**

2 Doxygen **4**

3 Testovi i pokrivenost koda **6**

3.1 Alat LCOV za analizu pokrivenosti koda 7

3.2 Unit testovi 8

3.3 Integracioni testovi 9

3.4 Pokrivenost koda nakon implementacije testova 9

4 Valgrind **11**

4.1 Memcheck 11

4.1.1 Rezultati Memcheck analize 12

4.2 Massif 13

4.2.1 Rezultati Massif analize 14

4.2.2 Poređenje rezultata Memcheck i Massif analize 15

5 Cppcheck **16**

5.1 Rezultati Cppcheck analize 16

6 Clang-tidy **17**

6.1 Rezultati Clang-Tidy analize 18

7 Zaključak **19**

1 Uvod

GuessMyDrawing je višekorisnička igra „Pogodi šta crtam“ realizovana kao klijent–server GUI aplikacija. Projekat je dostupan na sledećem Git repozitorijumu: `https://gitlab.com/matf-bg-ac-rs/course-rs/projects-2021-2022/07-GuessMyDrawing.git` .

Aplikacija je implementirana u programskom jeziku C++ uz upotrebu Qt5 biblioteka i Qt Widgets okruženja. Korisnici se povezuju na serverski deo aplikacije, gde imaju mogućnost da pristupe postojećim sobama ili da kreiraju novu sobu uz definisanje trajanja runde. U okviru svake sobe jedan igrač bira zadatu reč i crta na zajedničkom canvasu, dok ostali učesnici pokušavaju da pogode reč putem chat komunikacije.

Pre sprovođenja analize projekat je preuzet sa zvaničnog Git repozitorijuma, buildovan i pokrenut u skladu sa uputstvima navedenim od strane autora projekta.

Build

```
git clone https://gitlab.com/matf-bg-ac-rs/course-rs/projects-2021-2022/07-GuessMyDrawing.git
cd 07-GuessMyDrawing/07-GuessMyDrawing
mkdir build
cd build
qmake ..
make
```

Run

- Starting the server:

```
./Server/Server
```

- Starting the game:

```
./Application/Application
```

Slika 1: Preuzimanje, buildovanje i pokretanje projekta u skladu sa uputstvom autora

U radu je korišćeno više alata koji pripadaju različitim oblastima verifikacije softvera. Alat `Doxygen` korišćen je za generisanje dokumentacije izvornog koda i analizu strukture projekta. Za testiranje aplikacije implementirani su unit i integracioni testovi korišćenjem biblioteke Catch2, dok je alat `LCOV` korišćen za analizu pokrivenosti koda tokom izvršavanja testova.

Pored toga, korišćen je alat `Valgrind` za analizu upravljanja memorijom i otkrivanje potencijalnih curenja memorije tokom izvršavanja programa. Za statičku analizu izvornog koda korišćeni su alati `Cppcheck` i `Clang-Tidy`, koji omogućavaju pronalaženje potencijalnih grešaka, neefikasnosti i stilskih problema u kodu.

2 Doxygen

Doxygen je značajan u procesu verifikacije softvera jer generiše dokumentaciju direktno iz izvornog koda, uključujući opise klasa, funkcija, parametara i njihovih uloga u sistemu. Kada su struktura i namena koda jasno dokumentovane, razumevanje sistema postaje znatno jednostavnije: verifikator može brže identifikovati ključne module, analizirati međusobne zavisnosti i uočiti kritične tačke sistema.

Takvo razumevanje direktno doprinosi kvalitetnijem testiranju. Jasno definisani interfejsi i odnosi između komponenti olakšavaju izbor test slučajeva, pisanje testova i proveru graničnih situacija, jer je precizno definisano šta svaka funkcija očekuje kao ulaz i koje rezultate proizvodi.

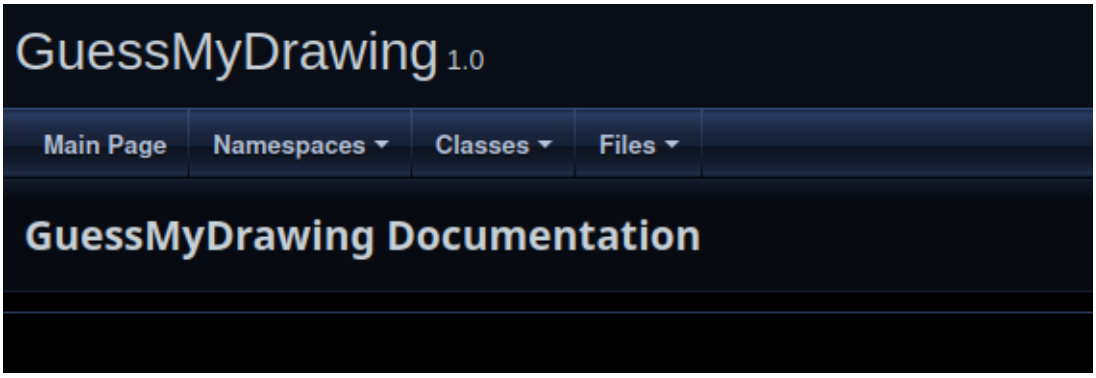
U radu je korišćena sledeća konfiguracija alata Doxygen:

Parametar	**Vrednost**	**Opis**
`PROJECT_NAME`	GuessMyDrawing	Naziv projekta koji se prikazuje u generisanoj dokumentaciji.
`PROJECT_NUMBER`	1.0	Verzija projekta.
`INPUT`	`$PROJECT_DIR`	Putanja do direktorija sa izvornim kodom.
`RECURSIVE`	YES	Omogućava pretraživanje svih poddirektorija.
`FILE_PATTERNS`	`*.cpp *.h *.hpp *.c`	Tipovi fajlova koje Doxygen analizira.
`OUTPUT_DIRECTORY`	`$RESULTS_DIR`	Direktorij u koji se generiše dokumentacija.
`GENERATE_HTML`	YES	Generiše HTML dokumentaciju.
`GENERATE_LATEX`	NO	Isključuje LaTeX dokumentaciju.
`EXTRACT_ALL`	YES	Dokumentuje sve elemente koda.
`EXTRACT_PRIVATE`	NO	Privatni članovi klasa nisu uključeni.
`EXTRACT_STATIC`	YES	Uključuje statičke funkcije i varijable.
`SOURCE_BROWSER`	YES	Omogućava pregled izvornog koda.
`REFERENCED_BY_RELATION`	YES	Prikazuje koje funkcije koriste određenu funkciju.
`REFERENCES_RELATION`	YES	Prikazuje koje funkcije određena funkcija poziva.
`QUIET`	NO	Doxygen ne radi u tihom režimu već prikazuje informacije tokom generisanja dokumentacije.
`WARNINGS`	YES	Omogućava prikaz upozorenja tokom generisanja dokumentacije.
`HAVE_DOT`	YES	Omogućava korišćenje Graphviz alata za generisanje dijagrama.
`CALL_GRAPH`	YES	Generiše graf poziva funkcija.
`CALLER_GRAPH`	YES	Generiše graf funkcija koje pozivaju datu funkciju.
`CLASS_DIAGRAMS`	YES	Generiše UML dijagrame klasa.

Tabela 1: Konfiguracija Doxygen alata

U ovom projektu konfiguracioni fajl se generiše automatski korišćenjem Bash skripte `run_doxygen.sh`, koja definiše potrebne opcije i pokreće Doxygen alat nad izvornim kodom projekta.

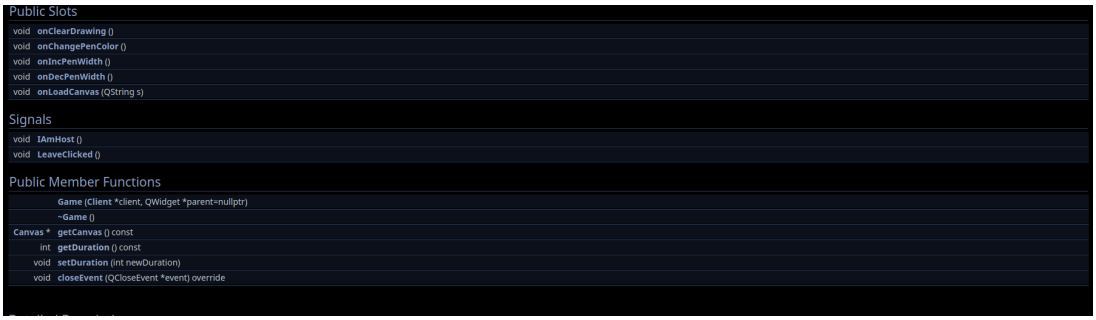
Nakon pokretanja Doxygen alata generiše se dokumentacija u HTML formatu. Glavna stranica dokumentacije nalazi se u fajlu `index.html` unutar generisanog direktorijuma. HTML dokumentacija omogućava pregled strukture projekta, uključujući liste klasa, funkcija, fajlova i njihovih međusobnih odnosa. Takođe su dostupni dijagrami klasa i grafovi poziva funkcija koji olakšavaju razumevanje strukture sistema.



Slika 2: Početna stranica generisane Doxygen dokumentacije

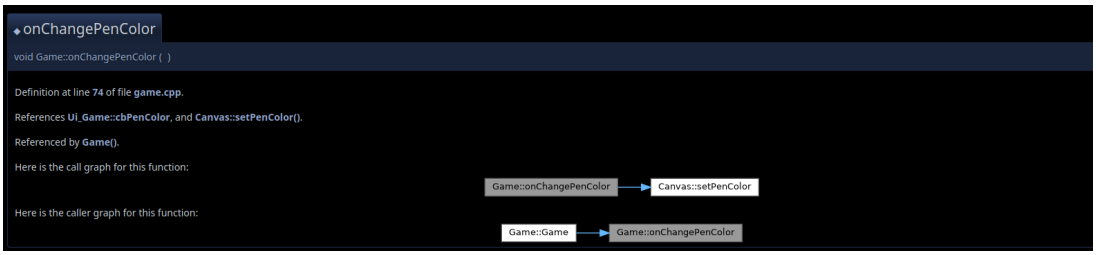
Na slici 2 prikazana je početna stranica generisane Doxygen dokumentacije. Početna stranica predstavlja centralno mesto za pregled dokumentacije projekta.

U slučaju da izvorni kod sadrži Doxygen komentare, na ovoj stranici bi bili prikazani opisi modula, klasa i funkcija sistema. U gornjem delu stranice nalazi se navigacioni meni koji omogućava pregled različitih delova dokumentacije, kao što su prostor imena (namespaces), klase i fajlovi projekta.



Slika 3: Prikaz klase Game u generisanoj Doxygen dokumentaciji

Na slici 3 prikazan je pregled klase `Game` u generisanoj Doxygen dokumentaciji. U okviru dokumentacije mogu se videti slotovi, signali i funkcije definisane u ovoj klasi. Za svaku funkciju prikazan je njen povratni tip i argumenti, što omogućava lakše razumevanje strukture i funkcionalnosti.



Slika 4: Graf poziva funkcije `onChangePenColor`

Na slici 4 prikazan je graf poziva slota `onChangePenColor`. Iz grafa se može videti koje funkcije pozivaju posmatrani slot, kao i koje funkcije on dalje poziva. U ovom slučaju slot `onChangePenColor` poziva funkciju `Canvas::setPenColor`, dok je sam slot pozvan iz konstruktora klase `Game`. Ovakav prikaz olakšava razumevanje toka izvršavanja programa i međusobnih zavisnosti između komponenti sistema.

3 Testovi i pokrivenost koda

Pokrivenost testiranjem (eng. *test coverage*) predstavlja metod za procenu u kojoj meri su različiti delovi softverskog sistema obuhvaćeni testovima. Ova metrika pomaže u proceni kvaliteta testnog skupa i omogućava identifikaciju delova sistema koji nisu dovoljno testirani. Na osnovu pokrivenosti moguće je steći uvid u to koliko testovi proveravaju funkcionalnost, strukturu i ponašanje sistema, kao i proceniti spremnost softvera za dalje korišćenje ili isporuku.

Pokrivenost se najčešće izražava kao procenat elemenata sistema koji su obuhvaćeni testiranjem u odnosu na ukupan broj elemenata te vrste u sistemu. Postoji više različitih vrsta pokrivenosti koje se koriste za procenu kvaliteta testiranja.

Pokrivenost zahteva (*requirements coverage*) predstavlja meru koja pokazuje koliki je procenat definisanih zahteva proveren testiranjem u odnosu na ukupan broj zahteva specificiranih u dokumentaciji sistema.

Pokrivenost funkcionalnosti (*functional coverage*) pokazuje koliki je procenat funkcionalnosti sistema obuhvaćen testovima, odnosno u kojoj meri su implementirane funkcije sistema proverene tokom testiranja.

Pokrivenost koda (*code coverage*) predstavlja skup metrika koje pokazuju koliki deo izvornog koda je izvršen tokom testiranja. Jedan od najčešće korišćenih pokazatelja je pokrivenost linija koda, koja se definiše kao odnos broja izvršenih linija koda tokom testiranja i ukupnog broja linija koda u projektu.

3.1 Alat LCOV za analizu pokrivenosti koda

Za analizu pokrivenosti koda korišćen je alat LCOV koji omogućava generisanje izveštaja o pokrivenosti linija i funkcija tokom izvršavanja testova. Na osnovu generisanog izveštaja moguće je utvrditi koji delovi koda su obuhvaćeni testovima, a koji nisu.

Izveštaj o pokrivenosti generisan je nakon pokretanja testova nad projektom, pri čemu LCOV prikuplja informacije o izvršavanju koda i generiše HTML izveštaj koji prikazuje statistiku pokrivenosti linija i funkcija.

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level	Lines:	41.8 %	5924	2476
Test:	coverage.info	Functions:	35.6 %	1210	431
Test Date:	2026-02-20 22:01:58				

Directory	Line Coverage			Function Coverage		
	Rate	Total	Hit	Rate	Total	Hit
07-GuessMyDrawing/Application	73.5 %	230	169	89.7 %	29	26
07-GuessMyDrawing/Server	65.2 %	348	227	77.6 %	49	38
tests	38.1 %	5275	2009	31.9 %	1123	358
tests/seminar_tests	100.0 %	71	71	100.0 %	9	9

Generated by: LCOV version 2.0-1

Slika 5: Ukupna pokrivenost koda generisana alatom LCOV

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level - 07-GuessMyDrawing/Application	Lines:	57.8 %	230	133
Test:	coverage.info	Functions:	51.7 %	29	15
Test Date:	2026-02-20 20:25:04				

Filename	Line Coverage			Function Coverage		
	Rate	Total	Hit	Rate	Total	Hit
client.cpp	23.1 %	117	27	26.3 %	19	5
client.h	100.0 %	1	1	-		
messageparser.cpp	93.8 %	112	105	100.0 %	10	10

Generated by: LCOV version 2.0-1

Slika 6: Pokrivenost koda za modul Application generisana alatom LCOV

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level - 07-GuessMyDrawing/Server	Lines:	65.2 %	348	227
Test:	coverage.info	Functions:	77.6 %	49	38
Test Date:	2026-02-20 22:01:58				

Filename	Line Coverage			Function Coverage		
	Rate	Total	Hit	Rate	Total	Hit
room.cpp	79.8 %	114	91	84.6 %	13	11
server.cpp	39.5 %	124	49	64.3 %	14	9
servermessageparser.cpp	89.0 %	73	65	100.0 %	12	12
thread.cpp	59.5 %	37	22	60.0 %	10	6

Slika 7: Pokrivenost koda za modul Server generisana alatom LCOV

Na osnovu izveštaja generisanog alatom LCOV može se uočiti da početna pokrivenost koda testovima nije na visokom nivou. Ukupna pokrivenost linija koda iznosi približno 41.8%, dok pokrivenost funkcija iznosi oko 35.6%. Analizom pojedinačnih modula može se primetiti da serverski deo aplikacije ima nešto bolju pokrivenost, gde pokrivenost linija dostiže oko 65.2%, dok je pokrivenost funkcija oko 77.6%. Sa druge strane, u aplikativnom delu projekta pokrivenost linija iznosi oko 57.8%, dok je pokrivenost funkcija približno 51.7%. Ovi rezultati ukazuju na to da značajan deo implementiranog koda nije obuhvaćen

testovima, što predstavlja prostor za unapređenje testnog skupa i dodatno proširenje testova u cilju postizanja veće pouzdanosti sistema.

3.2 Unit testovi

Testiranje jedinica koda (eng. *unit testing*) predstavlja proveru ispravnosti najmanjih delova softverskog sistema koji se mogu testirati nezavisno od ostatka aplikacije. U objektno orijentisanom programiranju ove jedinice su najčešće pojedinačne klase ili njihove metode, dok se u drugim programskim paradigmama testiraju funkcije, procedure ili moduli.

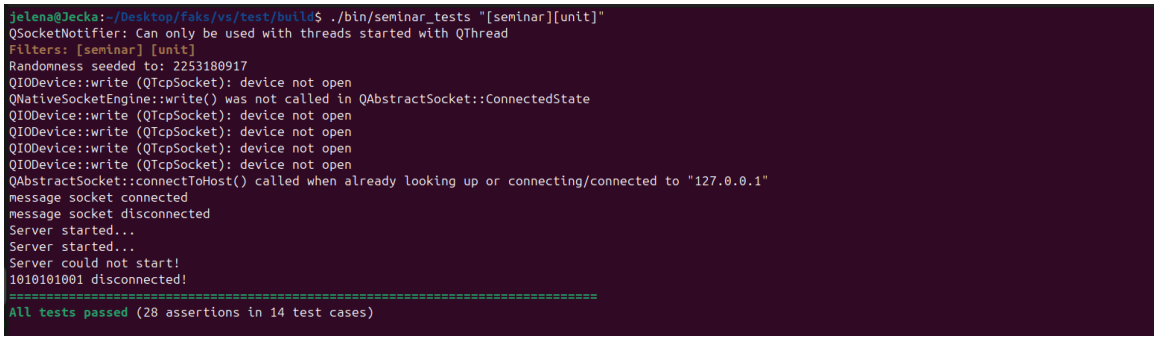
Cilj unit testiranja je da se proveri ispravnost ponašanja svake pojedinačne komponente sistema, čime se obezbeđuje stabilna osnova za dalju integraciju sa drugim delovima aplikacije. Ranim otkrivanjem grešaka u pojedinačnim komponentama olakšava se kasniji razvoj sistema i smanjuje mogućnost pojave kompleksnih problema tokom integracije različitih modula.

U okviru projekta implementirani su unit testovi za proveru ispravnosti pojedinačnih komponenti sistema. Testovi su napisani korišćenjem biblioteke Catch2 i fokusirani su na proveru osnovnih funkcionalnosti klasa koje čine klijentski i serverski deo sistema. Unit testovi obuhvataju klase `Client`, `Room`, `Server` i `Thread`. Testiranjem su proverene osnovne funkcionalnosti

Nakon implementacije unit testova, izvršeno je njihovo kompajliranje i pokretanje korišćenjem CMake sistema za izgradnju projekta. Najpre je kreiran direktorijum za build, nakon čega je projekat konfigurisan pomoću komande `cmake`, a zatim i kompajliran korišćenjem komande `make`.

Nakon uspešnog kompajliranja generisan je izvršni program `seminar_tests` koji sadrži sve definisane test slučajeve. Pokretanje unit testova vrši se izvršavanjem ovog programa uz filter `[seminar][unit]` koji omogućava izdvajanje samo jediničnih testova.

```
./bin/seminar_tests "[seminar][unit]"
```



```
jelena@Jecka:~/Desktop/faks/vs/test/build$ ./bin/seminar_tests "[seminar][unit]"
QSocketNotifier: Can only be used with threads started with QThread
Filters: [seminar] [unit]
Randomness seeded to: 2253180917
QIODevice::write (QTcpSocket): device not open
QNativeSocketEngine::write() was not called in QAbstractSocket::ConnectedState
QIODevice::write (QTcpSocket): device not open
QIODevice::write (QTcpSocket): device not open
QIODevice::write (QTcpSocket): device not open
QIODevice::write (QTcpSocket): device not open
QAbstractSocket::connectToHost() called when already looking up or connecting/connected to "127.0.0.1"
message socket connected
message socket disconnected
Server started...
Server started...
Server could not start!
1010101001 disconnected!
===============================================================================
All tests passed (28 assertions in 14 test cases)
```

Slika 8: Rezultat izvršavanja unit testova

Rezultat izvršavanja prikazan na slici 8 potvrđuje da implementirani unit testovi prolaze bez grešaka i da testirane funkcionalnosti rade u skladu sa očekivanim ponašanjem

sistema.Ukupno je izvršeno 14 test slučajeva sa 28 provera (assertions).Ukupno je izvršeno 14 test slučajeva sa 28 provera (assertions)

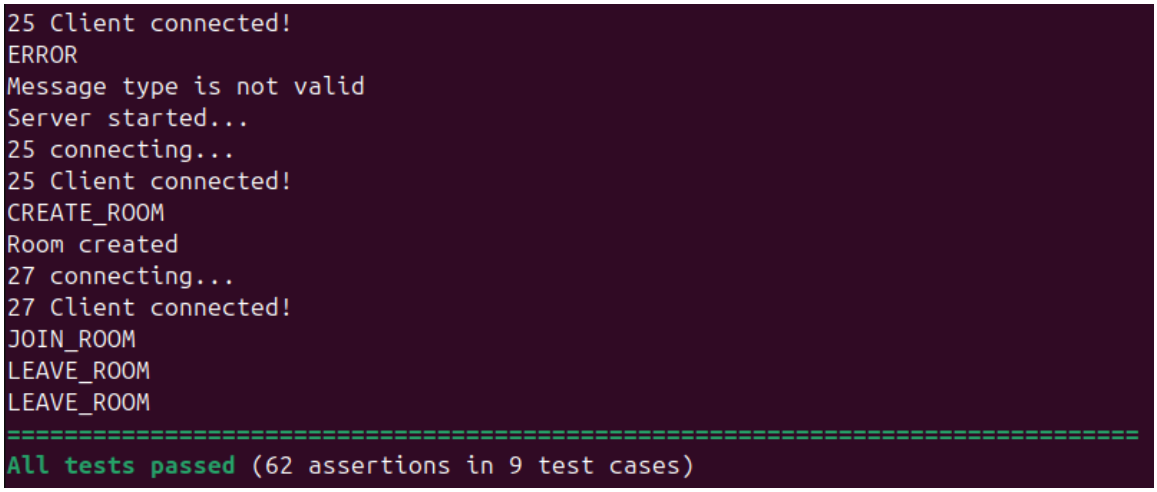
3.3 Integracioni testovi

Integracioni testovi predstavljaju testove koji proveravaju ispravnost saradnje između više komponenti sistema. Za razliku od unit testova, gde se pojedinačne klase testiraju izolovano, integracioni testovi proveravaju da li različiti delovi sistema ispravno komuniciraju i rade zajedno.

U okviru projekta implementirani su integracioni testovi koji proveravaju komunikaciju između klijenta i servera, odnosno uspostavljanje konekcije i razmenu poruka između komponenti aplikacije. Na ovaj način se proverava da li klijentski i serverski deo sistema pravilno sarađuju tokom izvršavanja aplikacije.

Integracioni testovi su takođe implementirani korišćenjem biblioteke Catch2. Kao i kod unit testova, nakon kompajliranja projekta moguće je pokrenuti samo integracione testove korišćenjem odgovarajućeg filtera.

```
./bin/seminar_tests "[seminar][integration]"
```



Slika 9: Rezultat izvršavanja integracioni testova

Rezultat 9 pokazuje da saradnja između klijentskog i serverskog dela sistema funkcioniše očekivano i da su testirani scenariji uspešno izvršeni.

3.4 Pokrivenost koda nakon implementacije testova

Analiza pokrivenosti koda pokreće se izvršavanjem skripte `run_coverage.sh`, koja automatizuje proces prikupljanja informacija o pokrivenosti. Pokretanjem ove skripte izvršavaju se testovi, prikupljaju se podaci o izvršenim delovima koda i generiše se HTML izveštaj koji omogućava pregled pokrivenosti linija i funkcija u projektu.

```
./run_coverage.sh
```

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level				
Test:	coverage.info	Lines:	79.4 %	748	594
Test Date:	2026-03-04 20:14:06	Functions:	71.1 %	194	138

Directory	Line Coverage			Function Coverage		
	Rate	Total	Hit	Rate	Total	Hit
Application	83.1 %	255	212	89.7 %	29	26
Server	83.4 %	391	326	92.5 %	53	49
/home/jelena/Desktop/faks/vs/build/_deps/catch2-src/src/catch2	14.3 %	42	6	3.2 %	31	1
/home/jelena/Desktop/faks/vs/build/_deps/catch2-src/src/catch2/internal	83.3 %	60	50	76.5 %	81	62

Generated by: LCOV version 2.0-1

Slika 10: Ukupna pokrivenost koda nakon implementacije testova

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level - Server				
Test:	coverage.info	Lines:	83.4 %	391	326
Test Date:	2026-03-04 20:14:06	Functions:	92.5 %	53	49

Filename	Line Coverage			Function Coverage		
	Rate	Total	Hit	Rate	Total	Hit
room.cpp	84.4 %	122	103	100.0 %	13	13
room.h	100.0 %	1	1	100.0 %	1	1
server.cpp	77.9 %	140	109	92.9 %	14	13
servermainwindow.cpp	0.0 %	7		0.0 %	3	
servermessageparser.cpp	91.6 %	83	76	100.0 %	12	12
thread.cpp	97.4 %	38	37	100.0 %	10	10

Generated by: LCOV version 2.0-1

Slika 11: Pokrivenost koda u serverskom delu aplikacije

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level - Application				
Test:	coverage.info	Lines:	83.1 %	255	212
Test Date:	2026-03-04 20:14:06	Functions:	89.7 %	29	26

Filename	Line Coverage			Function Coverage		
	Rate	Total	Hit	Rate	Total	Hit
client.cpp	89.8 %	137	123	89.5 %	19	17
messageparser.cpp	75.4 %	118	89	90.0 %	10	9

Generated by: LCOV version 2.0-1

Slika 12: Pokrivenost koda u klijentskom delu aplikacije

Nakon implementacije unit i integracionih testova ponovo je pokrenuta analiza pokrivenosti koda korišćenjem alata LCOV. Rezultati pokazuju da ukupna pokrivenost linija koda iznosi 79.4%, dok pokrivenost funkcija iznosi 71.1%.

Detaljniji pregled pokazuje da je pokrivenost koda u okviru serverskog dela sistema 83.4% za linije koda i 92.5% za funkcije. U okviru klijentskog dela aplikacije pokrivenost linija iznosi 83.1%, dok pokrivenost funkcija iznosi 89.7%.

Može se primetiti da pojedini delovi sistema, poput `servermainwindow.cpp`, nisu obuhvaćeni testovima, što je očekivano jer se radi o grafičkom interfejsu koji nije testiran automatizovanim testovima.

Na osnovu dobijenih rezultata može se zaključiti da implementirani testovi obuhvataju većinu ključnih delova aplikacije i značajno doprinose povećanju pokrivenosti koda i pouzdanosti sistema.

Važno je napomenuti da čak i visoka pokrivenost koda, pa čak i pokrivenost od 100%, ne garantuje da softver ne sadrži greške. Potpuna sigurnost u ispravnost programa mogla bi se postići jedino kada bi se ispitali svi mogući scenariji izvršavanja, što u praksi često nije izvodljivo. Zbog toga se u praksi određuje nivo pokrivenosti koji se smatra dovoljnim za određeni projekat, kao i kombinacija različitih vrsta testiranja koje je potrebno primeniti.

Zahtevi za pokrivenošću i obim testiranja razlikuju se u zavisnosti od vrste softverskog sistema. Na primer, sistemi visokog rizika, kao što su sistemi u automobilskoj industriji, medicinski sistemi ili bankarski softver, zahtevaju znatno strože procedure testiranja, veću pokrivenost koda i kombinaciju različitih metoda testiranja kako bi se obezbedio visok nivo pouzdanosti i bezbednosti sistema. Sa druge strane, u manje kritičnim aplikacijama, poput video igara, obično se ne zahteva toliko visok nivo pokrivenosti koda, već je fokus češće na testiranju funkcionalnosti i korisničkog iskustva.

4 Valgrind

Valgrind predstavlja platformu za dinamičku analizu programa koja funkcioniše na principu instrumentacije tokom izvršavanja aplikacije. U okviru platforme dostupno je više specijalizovanih alata, od kojih svaki koristi sopstvenu vrstu instrumentacije i namenjen je analizi različitih aspekata ponašanja programa.

Najpoznatiji alati u okviru Valgrind platforme su:

- **Memcheck** — detektor memorijskih grešaka.
- **Massif** — alat za analizu upotrebe dinamičke memorije i praćenje memorijskog zauzeća.
- **Cachegrind** — profajler keš memorije.
- **Callgrind** — profajler funkcija i analiza poziva funkcija.
- **Helgrind** i **DRD** — alati za detekciju grešaka u višenitnom izvršavanju programa.

4.1 Memcheck

U ovom radu korišćen je Memcheck, jedan od najpoznatijih i najčešće korišćenih alata u okviru Valgrind platforme. Njegova primarna uloga je detekcija grešaka u upravljanju memorijom, koje se prikazuju u izveštaju generisanom od strane Memcheck-a. Memcheck omogućava otkrivanje različitih tipova memorijskih nepravilnosti, kao što su upisivanje

podataka van granica alocirane memorije na hipu ili steku, pristupanje memoriji koja je već oslobođena, neispravno ili dvostruko oslobađanje memorije, korišćenje neinicijalizovanih vrednosti, kao i curenja memorije.

Da bi se koristio Memcheck alat, neophodno je da Valgrind bude instaliran na sistemu. Na Linux operativnim sistemima instalacija se vrši korišćenjem sledeće komande **sudo apt install valgrind** .

U ovom radu Memcheck je pokretan sa sledećim opcijama:

```
valgrind --leak-check=full \
         --show-leak-kinds=all \
         --track-origins=yes \
         --log-file="${RESULTS_DIR}/${OUTPUT_FILE}" \
         "$APP_BIN"
```

Slika 13: Pokretanje Memcheck alata sa korišćenim opcijama

Opcija `-leak-check=full` omogućava detaljnu analizu curenja memorije i prikazuje lokaciju svake alokacije koja nije pravilno oslobođena. Korišćenjem opcije `-show-leak-kinds=all` prikazuju se sve kategorije curenja memorije, uključujući definite, indirect, possible i reachable curenja. Opcija `-track-origins=yes` omogućava praćenje porekla neinicijalizovanih vrednosti, čime se olakšava identifikacija izvora potencijalnih grešaka. Konačno, opcija `-log-file` omogućava da se kompletan izveštaj analize sačuva u tekstualni fajl radi kasnije obrade i dokumentovanja rezultata.

Analiza je vršena nad prethodno buildovanim kodom projekta. Tok pokretanja aplikacije organizovan je na sledeći način: najpre se pokreće serverski deo aplikacije, nakon čega se pokreće klijentska aplikacija. Korisnik se potom povezuje na server, koristi aplikaciju, a po njenom zatvaranju Valgrind automatski generiše izveštaj o izvršenoj analizi.

Generisani izveštaj se upisuje u fajl `results/valgrind_report.txt`. Pokretanje kompletne procedure automatizovano je putem Bash skripte `run_valgrind.sh`, pri čemu se putanja do projekta može proslediti kao argument, dok je podrazumevana vrednost `../07-GuessMyDrawing-master`.

4.1.1 Rezultati Memcheck analize

U tabeli je prikazan izveštaj generisan alatom Memcheck:

Stavka iz izveštaja	**Vrednost**
in use at exit	17,950,553 bajta u 42,128 blokova (≈ 17.1 MB)
total heap usage	433,174 alokacija, 391,046 oslobađanja, 82,044,321 bajt alocirano (≈ 78 MB)
still reachable	17,741,065 bajtova u 41,106 blokova
definitely lost	960 bajtova u 12 blokova
indirectly lost	64 bajta u 2 bloka
possibly lost	113,528 bajtova u 145 blokova
ERROR SUMMARY	77 grešaka u 77 konteksta (suppressed: 0)

Na osnovu sprovedene analize alatom Memcheck može se zaključiti da aplikacija ne sadrži kritične memorijske greške poput nevažećih pristupa memoriji ili ozbiljnih curenja memorije koja bi mogla dovesti do nestabilnosti sistema.

Ukupna količina memorije označena kao *definitely lost* i *indirectly lost* iznosi veoma mali broj bajtova u odnosu na ukupnu količinu alocirane memorije tokom izvršavanja programa, što ukazuje na postojanje manjih memorijskih nepravilnosti koje ne utiču značajno na rad aplikacije. Deo memorije označen kao *still reachable* u najvećoj meri potiče iz Qt i sistemskih biblioteka i predstavlja očekivano ponašanje pri završetku GUI aplikacija.

Može se zaključiti da je memorijsko upravljanje u okviru aplikacije stabilno i da nema ozbiljnih problema, uz preporuku da se identifikovani manji memorijski gubici dodatno analiziraju i po potrebi optimizuju u daljem razvoju projekta.

4.2 Massif

Kao dopuna analizi memorije pomoću alata Memcheck, u okviru Valgrind platforme korišćen je i alat Massif. Massif je alat namenjen analizi korišćenja memorije u hip segmentu programa i omogućava praćenje načina na koji aplikacija alocira memoriju tokom izvršavanja.

Postoje određeni scenariji neefikasnog korišćenja memorije koje Memcheck ne može direktno da detektuje. Takve situacije nastaju kada memorija formalno nije izgubljena — pokazivač na nju i dalje postoji — ali se ta memorija više ne koristi tokom izvršavanja programa. U tom slučaju program i dalje zauzima resurse koji mu zapravo nisu potrebni.

Massif omogućava identifikaciju upravo takvih slučajeva, jer prikazuje koliko memorije program koristi tokom vremena i koji delovi koda su odgovorni za najveće alokacije. Na taj način alat pomaže u otkrivanju mesta u programu gde dolazi do neefikasnog upravljanja memorijom i omogućava lakšu optimizaciju upotrebe memorijskih resursa.

U ovom radu Massif je pokretan sa sledećim opcijama:

```
valgrind --tool=massif \
         --massif-out-file="${RESULTS_DIR}/${OUTPUT_FILE}" \
         --verbose \
         "$APP_BIN"
```

Slika 14: Pokretanje Massif alata sa korišćenim opcijama

Opcija `-tool=massif`, čime se aktivira Massif profilisanje memorije. Opcija `-massif-out-file` određuje fajl u koji se upisuje izveštaj analize, dok `-verbose` omogućava detaljniji prikaz informacija tokom izvršavanja alata.Dobijeni rezultat analize čuva se u fajlu `massif.out`, a njegov pregled i interpretacija vrše se pomoću komande `ms_print`. Ova komanda prikazuje grafikon potrošnje memorije kroz vreme, kao i detaljne informacije o snapshot-ovima i funkcijama koje su odgovorne za najveće alokacije memorije.

4.2.1 Rezultati Massif analize

U tabeli su prikazane najvažnije vrednosti dobijene iz Massif izveštaja.

Stavka iz izveštaja	**Vrednost**
Broj snapshot-a	68
Peak snapshot	56
Ukupna memorija u peak trenutku	24,414,368 B ($\approx$ 23.3 MB)
Korisni heap (useful-heap)	23,662,669 B
Dodatni heap overhead	751,699 B

Massif grafikon pokazuje promenu potrošnje heap memorije tokom izvršavanja programa, pri čemu se najveća potrošnja javlja u snapshot-u označenom kao *peak*. U analiziranom slučaju maksimalna potrošnja memorije iznosila je približno 23.3 MB.

Detaljna analiza pozivnog stabla u peak snapshot-u pokazuje da najveći deo memorije potiče iz Qt biblioteka, posebno iz funkcija povezanih sa klasom `QImage`. Oko 74% ukupne heap memorije u tom trenutku alocirano je u funkcijama `QImageData::create` i `QImage::QImage`, što odgovara baferima za crtanje koji se koriste u grafičkom interfejsu aplikacije.

Značajan deo alokacija u ranijim fazama izvršavanja povezan je sa inicijalizacijom grafičkog okruženja i biblioteka (Qt, Wayland i GLib), što je očekivano ponašanje za GUI aplikacije.

Na osnovu rezultata može se zaključiti da aplikacija ne pokazuje nekontrolisani rast potrošnje memorije tokom izvršavanja, već da najveći deo zauzeća potiče iz grafičkih

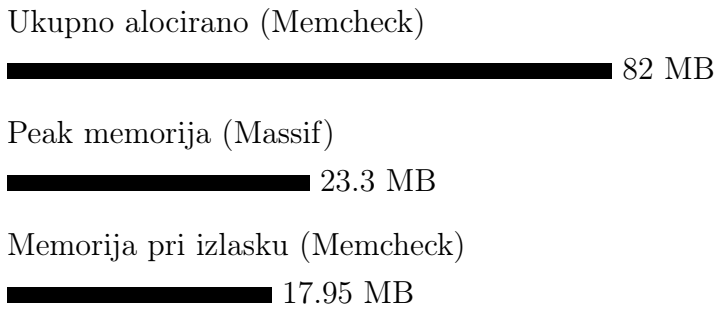
komponenti i inicijalizacije biblioteka. Time se potvrđuje da aplikacija koristi memoriju u okviru očekivanih granica za GUI program zasnovan na Qt platformi.

4.2.2 Poređenje rezultata Memcheck i Massif analize

Rezultati Memcheck i Massif alata međusobno se dopunjuju i zajedno daju potpuniju sliku o načinu na koji aplikacija upravlja memorijom. Memcheck je pokazao da pri završetku programa u upotrebi ostaje približno 17.95 MB memorije raspoređene u 42,128 blokova, pri čemu se u pozivnim lancima pojavljuju i delovi aplikativnog koda, kao što su `Client::Client`, `MainWindow::MainWindow` i `main`. Ako se ovaj nalaz uporedi sa rezultatima Massif analize, vidi se da maksimalna potrošnja heap memorije tokom izvršavanja dostiže približno 23.3 MB. U peak snapshot-u dominantan deo zauzeća potiče iz Qt biblioteka i inicijalizacije grafičkog okruženja, uključujući Qt, Wayland i GTK. Zajedničko pojavljivanje pozivnog lanca koji se završava u funkciji `main` ukazuje da su i Memcheck-ovi *still reachable* zapisi i Massif-ov peak povezani sa istim tokom izvršavanja aplikacije, odnosno sa pokretanjem GUI dela sistema i njegovim radom.

Memcheck je takođe pokazao da je tokom celokupnog izvršavanja programa izvršeno 433,174 alokacija, pri čemu je ukupno alocirano oko 82 MB memorije. Sa druge strane, Massif ne prikazuje ukupan broj alokacija, već pokazuje koliko je memorije zauzeto u određenim trenucima izvršavanja. U peak snapshot-u zabeleženo je oko 23.3 MB zauzeća heap memorije, pri čemu oko 74.15% te vrednosti potiče iz Qt funkcija `QImageData::create` i `QImage::QImage`.

Ovakav nalaz ukazuje da značajan deo memorijske potrošnje u vrhuncu rada aplikacije otpada na grafičke bafere za crtanje, odnosno na objekte povezane sa canvas komponentom igre. Na taj način Massif preciznije pokazuje gde se memorija najviše troši, dok Memcheck daje širu sliku o ukupnom broju alokacija i stanju memorije na kraju izvršavanja. Dodatno, Memcheck je pokazao da najveći deo memorije označene kao *still reachable* potiče iz sistemskih i Qt biblioteka, uključujući GTK, GLib i Qt komponente. Upoređivanjem sa Massif rezultatima može se uočiti da i u peak snapshot-u dominiraju upravo iste grupe biblioteka: Qt funkcije povezane sa klasom `QImage`, zatim Wayland/GTK inicijalizacija i GLib alokacije. Ovo potvrđuje da i memorija koja ostaje pri izlasku iz programa i memorija koja je dominantna u trenutku najveće potrošnje velikim delom potiču iz framework-a i grafičkog okruženja, a ne isključivo iz samog aplikativnog koda.



5 Cppcheck

Cppcheck je alat za statičku analizu C i C++ izvornog koda. Za razliku od alata za dinamičku analizu, Cppcheck ne izvršava program, već analizira izvorni kod i identifikuje potencijalne greške, nebezbedne konstrukcije, probleme stila, performansi i sumnjive obrasce u implementaciji.

Alat omogućava detekciju različitih tipova problema, kao što su curenje memorije, korišćenje neinicijalizovanih promenljivih, nepravilno rukovanje pokazivačima, mogući prelazak granica bafera, kao i preporuke za unapređenje kvaliteta koda (na primer, prosleđivanje parametara po referenci ili korišćenje kvalifikatora `const`).

Opcija	**Namena**
`-enable=all`	Uključuje sve kategorije provera (warning, style, performance, portability, information).
`-inconclusive`	Prikazuje i potencijalne (nepotvrđene) probleme.
`-std=c++17`	Definiše C++ standard korišćen tokom analize.
`-force`	Forsira analizu i u složenijim konfiguracijama projekta.
`-library=qt`	Omogućava prepoznavanje Qt tipova i struktura.
`-I`	Dodavanje direktorijuma sa zaglavljima radi pravilnog parsiranja projekta.
`-D__linux__, -DQ_OS_LINUX`	Definisanje makroa za Linux okruženje.
`-suppress=missingIncludeSystem`	Suzbija upozorenja vezana za sistemske include fajlove.
`-i build`	Isključuje build direktorijum iz analize.

5.1 Rezultati Cppcheck analize

Tip	**Broj**	**Opis**
style	46	Stilske preporuke (const, explicit, senčenje promenljivih, nekorišćene funkcije, hvatanje izuzetaka).
performance	28	Preporuke za optimizaciju (prosleđivanje po referenci, static funkcije).
warning	3	Potencijalni problemi (neinicijalizovana promenljiva, nedostajući copy konstruktor i operator=).
error	1	ODR konflikt generisanih Qt UI klasa.
information	4	Informativne poruke i neusklađene suppression direktive.

Najznačajniji nalazi Cppcheck analize odnose se na sledeće delove projekta:

- **uninitMemberVar** – U fajlu `Server/room.cpp` članska promenljiva `noMorePlayers` nije inicijalizovana u konstruktoru klase `Room`. Ovakva situacija može dovesti do nedefinisanog ponašanja programa.

- **noCopyConstructor** i **noOperatorEq** – U klasi `Room` (Server/room.cpp) vrši se dinamička alokacija memorije (`new ServerMessageParser()`), ali nisu definisani copy konstruktor i operator dodele. U slučaju kopiranja objekta može doći do problema poput dvostrukog oslobađanja memorije ili curenja.

- **noExplicitConstructor** – U fajlovima `Application/canvas.h`, `Application/mainwindow.h` i `Server/servermainwindow.h` konstruktori sa jednim parametrom nisu označeni ključnom rečju `explicit`, što može dozvoliti nenamerne implicitne konverzije tipova.

- **catchExceptionByValue** – U fajlu `Server/server.cpp` izuzeci tipa `QString` hvataju se po vrednosti umesto po referenci. Preporuka je korišćenje `catch (const QString&)` radi efikasnosti i pravilnog hvatanja izvedenih tipova.

- **shadowVariable** – U fajlu `Server/room.cpp` lokalne promenljive `name` i `message` zasenjuju postojeće promenljive istog imena, što može otežati razumevanje koda.

- **passedByValue** – U više fajlova u okviru Application i Server modula parametri tipa `QString` prosleđuju se po vrednosti umesto po `const` referenci, što može dovesti do nepotrebnog kopiranja objekata.

- **ctuOneDefinitionRuleViolation** – U generisanim fajlovima `build/Application/ui_mainwindo` i `build/Server/ui_servermainwindow.h` pojavljuju se dve klase sa imenom `Ui_MainWindow`. Cppcheck ovo prijavljuje kao kršenje ODR pravila, ali u praksi se radi o Qt generisanom kodu i ne predstavlja realnu grešku implementacije.

6 Clang-tidy

Clang-Tidy je alat za statičku analizu C i C++ koda, deo LLVM/Clang infrastrukture. Njegova namena je otkrivanje potencijalnih grešaka, stilskih nepravilnosti i problema u dizajnu, kao i davanje preporuka za modernizaciju i unapređenje kvaliteta koda.

Za razliku od kompajlera, Clang-Tidy primenjuje skup dodatnih pravila (checks) čime omogućava dublju analizu strukture programa. Precizno locira nepravilnosti, a za veliki broj njih nudi i mogućnost automatskog ispravljanja koda (tzv. *fixes*), čime se ubrzava proces refaktorisanja i unapređenja kvaliteta koda.

Analiza pomoću alata Clang-Tidy izvršena je nad kompletnim izvornim kodom projekta, pri čemu su iz analize izuzeti automatski generisani fajlovi u `build` direktorijumu. U analizi su korišćene grupe provera `clang-diagnostic` (kompajlerske dijagnostike),

`clang-analyzer` (otkrivanje potencijalnih logičkih i memorijskih grešaka), `modernize` (preporuke za savremenije C++ konstrukcije), `performance` (optimizacija koda), `readability` (poboljšanje čitljivosti), `bugprone` (česti izvori grešaka) i `cppcoreguidelines` (usklađenost sa C++ smernicama), čime je obuhvaćen širok spektar potencijalnih problema u kodu.

Analiza je fokusirana na module `Application` i `Server`, dok su sistemske i Qt biblioteke filtrirane kako bi se izbegli nerelevantni nalazi. Rezultati su generisani u tekstualnom izveštaju, sa preciznom lokacijom svake nepravilnosti u izvornom kodu.

6.1 Rezultati Clang-Tidy analize

Provera	**Broj**	**Opis**
modernize-use-trailing-return-type	473	Preporuka da se koristi sintaksa `auto f(...) -> Tip` umesto klasične deklaracije povratnog tipa.
readability-identifier-length	253	Identifikatori kraći od 3 karaktera (npr. ID, a, w, s).
nodiscard / modernize-use-nodiscard	144	Preporuka da funkcije čiji povratni tip treba koristiti budu označene sa `[[nodiscard]]`.
cppcoreguidelines-special-member-functions	90	Klasa ima destruktor ali nema copy/move specijalne funkcije.
cppcoreguidelines-owning-memory	62	Preporuke vezane za upravljanje dinamičkom memorijom (raw pokazivači).
performance-avoid-endl	31	Preporuka korišćenja '\n' umesto `std::endl`.
performance-enum-size	26	Enum koristi veći osnovni tip nego što je potrebno.

Tabela 2: Pregled najčešćih Clang-Tidy nalaza

Napomena: Isti nalaz može biti prikazan više puta u izveštaju, jer se zaglavlja (.h fajlovi) analiziraju u kontekstu svakog .cpp fajla koji ih uključuje. Na primer, fajl `thread.h` se proverava prilikom obrade svakog izvornog fajla koji ga uključuje, što dovodi do ponavljanja istih upozorenja. Zbog toga ukupan broj prijavljenih nalaza (1367) ne odgovara broju jedinstvenih lokacija u izvornom kodu.

Detaljnijom analizom uočeno je da se preporuke razlikuju po modulima projekta. U serverskom delu (thread.cpp i thread.h) izdvajaju se preporuke za inicijalizaciju članskih promenljivih u listi inicijalizatora konstruktora, prosleđivanje parametara po `const` referenci (npr. `QJsonObject`), zamenu `std::endl` sa `'\n'` radi boljih performansi, kao i označavanje metoda koje ne menjaju stanje objekta kao `const`. Takođe je sugerisano korišćenje `nullptr` umesto literala `0`, kao i eksplicitno definisanje ili zabrana copy/move specijalnih članova klase.

U okviru klase `Room` preporučeno je definisanje posebnih članova funkcija (copy i move

konstruktor, operator dodele), kao i preciznije imenovanje parametara radi bolje čitljivosti. U klasama `Server` i `MainWindow` predloženo je označavanje destruktora sa `override` i modernizacija zapisa povratnog tipa funkcija.

U aplikativnom delu projekta (messageparser, client, mainwindow, settings) najveći broj nalaza odnosi se na modernizaciju sintakse funkcija, označavanje metoda kao `const`, optimizaciju prosleđivanja parametara, kao i preporuke vezane za veličinu osnovnog tipa enumeracija poruka.

U zajedničkim zaglavljima identifikovani su nalazi vezani za kratke identifikatore (npr. `ID`) i optimizaciju tipa enumeracija.

7 Zaključak

U ovom radu izvršena je analiza kvaliteta softverskog projekta korišćenjem različitih alata za verifikaciju i analizu koda. Najpre je korišćen alat `Doxygen` za generisanje dokumentacije izvornog koda, čime je omogućen pregled strukture projekta, klasa i međusobnih odnosa između funkcija, kao i lakše razumevanje organizacije sistema.

U okviru testiranja implementirani su dodatni unit i integracioni testovi korišćenjem biblioteke Catch2. Ovi testovi obuhvataju ključne komponente sistema, uključujući klijentski i serverski deo aplikacije. Analizom pokrivenosti koda pomoću alata `LCOV` pokazano je da implementirani testovi značajno povećavaju pokrivenost koda. Nakon dodavanja testova ukupna pokrivenost linija koda iznosi približno 79%, dok pokrivenost funkcija iznosi oko 71%. Posebno je visoka pokrivenost u serverskom i klijentskom delu aplikacije, dok pojedini delovi koji se odnose na grafički interfejs nisu obuhvaćeni automatizovanim testovima.

Pored testiranja funkcionalnosti, izvršena je i analiza upravljanja memorijom pomoću alata `Valgrind`. Rezultati analize pokazali su da tokom izvršavanja programa nisu uočena značajna curenja memorije, što ukazuje na pravilno upravljanje dinamički alociranom memorijom u aplikaciji.

Za statičku analizu koda korišćeni su alati `Cppcheck` i `Clang-Tidy`. Ovi alati omogućavaju identifikaciju potencijalnih grešaka, stilskih problema i neefikasnosti u kodu. Rezultati analize pokazali su da u projektu nije pronađen veći broj kritičnih problema, već uglavnom manja upozorenja koja se odnose na stil koda ili potencijalna poboljšanja implementacije.

Na osnovu sprovedene analize može se zaključiti da kombinovanje različitih metoda verifikacije, kao što su testiranje, analiza pokrivenosti koda, statička analiza i analiza memorije, predstavlja važan korak u procesu razvoja kvalitetnog softvera. Ovakav pristup omogućava pravovremeno otkrivanje potencijalnih problema i doprinosi razvoju pouzdanijih i stabilnijih softverskih sistema.

Literatura

[1] Milena Vujošević Janičić. *Verifikacija softvera*. Matematički fakultet, Univerzitet u Beogradu.

[2] Materijali sa vežbi iz kursa *Verifikacija softvera*, Matematički fakultet, Univerzitet u Beogradu.

[3] Doxygen Documentation. Available at: `https://www.doxygen.nl/manual/index.html`

[4] Cppcheck Documentation. Available at: `https://cppcheck.sourceforge.io/manual.html`